REFONTE INTERNATIONAL TRAINING AND INTERNSHIP PROGRAM

Session 6: Model Based Development and Auto Code Generation

Spacecraft Software Engineer Program

Student: Tran Manh Truong **Module:** Advanced Application Phase, Unit 1

Schedule: Tue / Thu / Fri, 09:00 to 10:30 UTC **Builds on:** Session 5 mode logic

Up to now you have written flight software by hand. Much of the European space industry does not, at least not for control laws and mode logic. Instead the behaviour is modelled, the model is verified, and the C code is generated from it. This session is about what that changes: what you gain, what you give up, and why a generated file still has to be justified to a reviewer. The practical work takes your Lab 5.3 mode state machine and rebuilds it as a model, then proves the two versions behave identically.

Session Objectives

1. Explain why model based development is used for flight software and where it is not appropriate.
2. Specify a state machine with unambiguous execution semantics, rather than an informal diagram.
3. Generate C code from a model and assess it against flight coding constraints.
4. Perform back to back testing between a generated implementation and a hand written one.
5. Build a traceability chain from requirement to model element to generated code to test case.
6. Describe what tool qualification means and why a code generator does not remove verification obligations.

Tool access, read this first. Simulink with Stateflow and Embedded Coder, and SCADE, are commercial licensed products. SCADE in particular is not realistically available to an individual learner. Do not purchase anything for this session. Confirm with me what licensing Refonte can provide before you start, and in the meantime work Path B below, which is fully open source and teaches the same principles from the inside. Every objective above is reachable on Path B. The tool is the vehicle, not the subject.

Part A - Concepts (guided, approximately 45 minutes)

A1. What model based development actually is

In a model based flow, the design artefact is an executable model rather than source code. The model can be simulated before any target hardware exists, checked against the requirements, and then translated mechanically into C by a code generator.

What the industry is buying with this:

- **Early verification.** You can exercise the logic in simulation long before the onboard computer is available, which for a space programme is often years.
- **Fewer hand coding defects.** Transcription errors between a design document and C code largely disappear.
- **Traceability.** A good toolchain links requirement, model element, generated line, and test, which is exactly the evidence a space software review demands.
- **Consistency across a large team.** The model is the single source of truth, not one engineer's interpretation of a document.

A2. What it costs, stated honestly

- **Generated code is not written for humans.** It is correct but often unidiomatic, and debugging it means mapping back to the model rather than reading it directly.
- **The model becomes the thing you must get right.** A modelling error propagates perfectly into code. Generation guarantees faithfulness to the model, not correctness of the design.
- **Tool lock in and cost.** Licences are expensive and the artefacts are not portable between vendors.
- **Not everything suits it.** Control laws and state machines model well. Device drivers, bootloaders, and low level data handling usually do not, and are still hand written. A real onboard software build is a mixture.

A3. Semantics, the part that matters most

A state machine drawn on a whiteboard is ambiguous. A state machine you can generate code from must answer, precisely:

- What happens when two transitions out of one state are both enabled? What determines priority?
- Are transitions evaluated on an event, or on a periodic cycle, or both?
- Do entry and exit actions run on a self transition?
- When are inputs sampled, and are they stable for the whole cycle?
- Is the step function guaranteed to terminate, with a bounded number of transitions per cycle?

Why this matters for your Lab 5.3 code. Your hand written mode machine answered these questions implicitly, through the order of your if statements. That is normal and it worked. The difference in a model based flow is that the answers become explicit and reviewable, which is also how you find the cases you had not thought about. When you re specify it in Lab 6.1, expect to discover at least one ambiguity in your original.

A4. Code generation and flight coding constraints

Generated code still has to satisfy the same constraints as hand written flight code. Typical configuration requirements:

- No dynamic memory allocation. Static allocation only.
- Bounded loops, no recursion.
- Integer arithmetic preferred over floating point where the target lacks a reliable FPU, with explicit overflow behaviour.
- No unreachable or dead code in the delivered build, since it cannot be tested and therefore cannot be justified.
- Conformance to a coding standard, commonly a MISRA C subset for embedded and safety related work.

Generators are configurable for most of this, and the configuration itself becomes a controlled project artefact. Changing a generator setting is a design change, not a preference.

A5. Tool qualification

A natural question: if the generator is trustworthy, can you skip verifying the output? The answer in safety related domains is no, unless the tool has been qualified for that use, and even then the obligation is reduced rather than removed.

The reasoning is that a code generator is software too, and it can contain defects. So projects either qualify the tool with evidence from the vendor, or verify the output, or most commonly do a measure of both. The practical consequence for you: **back to back testing of generated code against an independent implementation is a standard technique**, and it is exactly what Lab 6.3 asks you to do.

A6. Model coverage and code coverage

Two distinct measures, both needed. Model coverage asks whether your simulation exercised every state, transition, and decision in the model. Code coverage asks the same of the generated source. They are not equivalent: a generator can emit code structures with no direct model counterpart, and uncovered generated code is a question you have to answer, usually by showing it is unreachable by construction or by adding a test.

Part B - Practical Labs

Lab 6.1 - Formal specification of the mode state machine approx 90 min

Goal: convert your Lab 5.3 mode logic from code into an explicit, tool independent model specification. This lab is required on both paths and is the foundation for the rest.

Write the machine as structured data, for example YAML or JSON, capturing states, events, guards, actions, and transition priority:

```
states:
  - name: NOMINAL
    on_entry:  [payload_enable, hk_rate_normal]
  - name: DEGRADED
    on_entry:  [payload_suspend, hk_rate_high, monitors_tighten]
```

```
  - name: SAFE
    on_entry:  [payload_off, schedule_suspend, hk_minimal, rx_guarantee]

transitions:
  - from: NOMINAL
    to: SAFE
    event: fault_confirmed
    guard: severity == CRITICAL
    priority: 1
  - from: NOMINAL
    to: DEGRADED
    event: fault_confirmed
    guard: severity == NON_CRITICAL
    priority: 2
  - from: SAFE
    to: NOMINAL
    event: tc_exit_safe_mode
    guard: ground_authorised
    priority: 1

semantics:
  evaluation: cyclic           # evaluated once per supervisor period
  input_sampling: start_of_cycle
  max_transitions_per_cycle: 1
  self_transition_runs_entry: false
```

What to produce alongside it

1. A written statement of the execution semantics, answering every question in A3 above.
2. A list of any ambiguity you found in your Lab 5.3 implementation while doing this. I expect at least one. Finding them is the point of the exercise, not a sign the earlier work was poor.
3. A completeness check: for every state, is every event either handled or explicitly declared as ignored? Unhandled events are where real state machines fail.

Lab 6.2 - Code generation approx 2 to 3 hours

Goal: produce C from the model of Lab 6.1. Take whichever path your tool access allows. The deliverable is the same either way.

Path A, commercial toolchain (only if Refonte provides licences)

1. Rebuild the Lab 6.1 machine in Stateflow, or in SCADE if available.
2. Simulate against a sequence of input scenarios and confirm the transitions match your specification.
3. Configure the generator for embedded targets: static allocation, no dynamic memory, integer types where possible, and your chosen coding standard checks.
4. Generate C, then run the toolchain's model advisor or equivalent static checks and record the findings.

5. Inspect the generated source and document how model elements map to generated functions and variables.

Path B, open source, and the default

Write your own generator. This sounds like a detour and it is not. Building a generator teaches you precisely what a commercial one does and why its output looks the way it does, and it is fully reproducible, free, and demonstrable to anyone looking at your repository.

1. Write a Python generator that reads the Lab 6.1 model file and emits a C module: a state enum, an event enum, a transition table, and a `step()` function.
2. Emit static allocation only, no dynamic memory, and a bounded step function with a single transition per cycle.
3. Emit a header with the public interface, and a generated banner marking the file as generated with the source model name and a hash of it, so a reviewer can tell it is not hand edited.
4. Emit entry and exit action hooks as declared functions the application implements, so the generated code contains logic and no policy.
5. Integrate the generated module into your RTEMS application in place of the Lab 5.3 hand written machine, and confirm it runs on QEMU.

A useful constraint to impose on yourself: the generated file must never be edited by hand. If something is wrong, fix the model or the generator and regenerate. This discipline is the whole point of the flow, and breaking it is the most common way real projects lose the benefit.

Lab 6.3 - Back to back equivalence testing approx 2 hours

Goal: demonstrate that the generated implementation and your hand written Lab 5.3 implementation behave identically, and investigate any case where they do not.

Method

1. Build a harness that drives both implementations with the same input sequence and records the state and actions produced by each after every cycle.
2. Generate test sequences covering: every transition at least once, every guard both true and false, unhandled events in each state, and simultaneous enabling of competing transitions.
3. Add a randomised sequence of several thousand events as a soak test.
4. Record every divergence and diagnose it. A divergence is a finding, not a failure of the lab.

What a divergence usually means. Most often the hand written version had an implicit behaviour the specification did not capture, which makes the specification incomplete rather than the code wrong. Occasionally the hand written version has a genuine bug that the formalisation exposed. Either outcome is a good result and both are worth writing up, so record which one each divergence was.

Coverage

Measure statement and branch coverage of the generated module under your test suite, using gcov or equivalent on a host build. Report uncovered lines and, for each, state whether it is unreachable by construction or untested. Unreachable by construction needs an argument, not an assertion.

Lab 6.4 - Traceability matrix approx 60 min

Goal: produce the evidence artefact that makes the above reviewable.

Build a table linking each mode management requirement to its model element, the generated code it produced, and the test case that exercises it:

Req ID	Requirement	Model element	Generated artefact	Test case	Result
MM-010	SAFE shall be exited only by ground telecommand	transition SAFE to NOMINAL, guard ground_authorised	`mode_step()` case SAFE	TC-SAFE-EXIT-01 to 03	to be filled
MM-020	Entry to SAFE shall suspend the onboard command schedule	SAFE on_entry schedule_suspend	`mode_enter_SAFE()`	TC-SAFE-ENTRY-02	

Write the requirements yourself, derived from your Lab 5.3 design note, and give them identifiers. Roughly eight to twelve requirements is the right scope. Deriving requirements from an existing implementation is backwards from real practice, and I want you to notice that: write one short paragraph on what that reversal makes difficult, since it is a real phenomenon on projects that code first and document later.

Assignment and Deliverables

```
labs/session-06/
  lab-6.1-model/           model file, semantics statement, ambiguities found
  lab-6.2-generation/      generator source (Path B) or model and config (Path A),
                           plus the generated module
  lab-6.3-equivalence/     harness, test sequences, divergence log, coverage report
  lab-6.4-traceability/    requirements list and traceability matrix
  docs/mbd-report.md       report (see below)
```

Written deliverable: mbd-report.md

Two to three pages:

1. Which path you took and why, stated plainly.
2. Your execution semantics, and the ambiguities Lab 6.1 exposed in the hand written version.
3. How model elements map to generated code.
4. Equivalence results: divergences found, cause of each, and which side was at fault.

5. Coverage results with justification for anything uncovered.
6. An assessment section: for this specific mode logic, was model based development worth it compared to the hand written version? Argue a position either way. A well argued "not worth it at this scale" is a better answer than an unexamined endorsement, and the ability to make that judgement is what distinguishes an engineer from a tool operator.

Submission

- Push to `labs/session-06/` and reply on Slack with the commit link.
- Include your divergence log in the message, even if it is empty, since an empty log is itself a result worth discussing.
- Confirm which path you took before you start, so I can support you on the right one.

Estimated effort: approximately 6 hours on Path B, somewhat less on Path A once licensing is sorted. As always an estimate for planning, not a target.

Reference Material

- **ECSS-E-ST-40C**, space software engineering, for the lifecycle and verification obligations that a model based flow still has to satisfy.
- **ECSS-Q-ST-80**, software product assurance, relevant to tool usage and output verification.
- **MISRA C guidelines**, for the coding standard discussion in A4.
- **MathWorks documentation** for Stateflow and Embedded Coder, if you are on Path A. The vendor documentation is the only reliable source for generator configuration.
- **gcov documentation**, for the coverage measurement in Lab 6.3.

On references and tools. As in previous sessions, I have no live access to search or a standards database here, so treat document designations as pointers to verify against the current official issue. The same applies to tool specifics: vendor interfaces and option names change between releases, so follow the documentation for your installed version rather than this document.

Looking Ahead to Session 7

Next is software verification and validation proper: the IVV and ISVV distinction, test levels, qualification evidence, and continuous integration for flight software. Your coverage work and traceability matrix from this session are the first pieces of that evidence set, and Session 7 assembles the rest of it around them.